



ABADDON

Gerador de Senhas Criptograficamente Seguras

O **Abaddon** é um gerador de senhas criptograficamente seguras desenvolvido com foco em alta entropia, resistência a ataques de força bruta e transparência dos métodos utilizados para geração de credenciais.

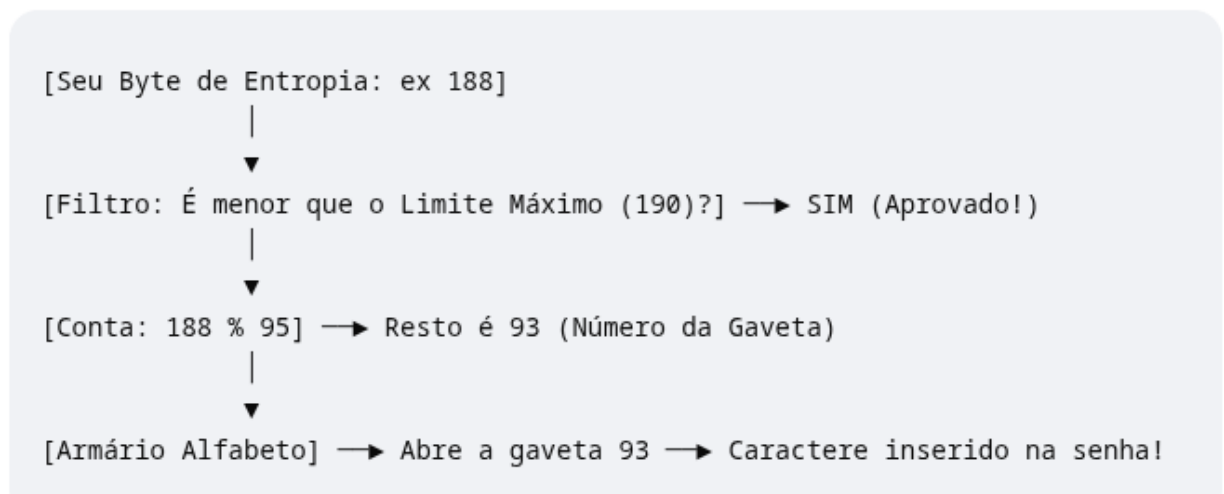
Entropia prevista por caractere:

<i>*Apenas números(10)</i>	<i>3,32 bits</i>
<i>*Letras minúsculas(26)</i>	<i>4,70 bits</i>
<i>*Letras + números(62)</i>	<i>5,95 bits</i>
<i>*ASCII imprimíveis(95)</i>	<i>6,57</i>

Base para Entropia Real Baseada em Silício

O projeto utiliza como fundamento fontes de entropia fornecidas pelo próprio sistema operacional, derivadas de eventos físicos observados pelo hardware do computador. Essas fontes incluem fenômenos eletrônicos e processos imprevisíveis coletados pelo kernel, permitindo a obtenção de números pseudoaleatórios criptograficamente seguros.

O Fluxo Visual do seu Pensamento



Características Principais

- Geração de senhas utilizando fontes de aleatoriedade criptograficamente seguras do sistema operacional.
- Suporte a diferentes conjuntos de caracteres, incluindo números, letras maiúsculas, letras minúsculas e símbolos especiais.
- Cálculo da força teórica da senha com base no universo de caracteres selecionado e no comprimento da senha.
- Implementação voltada para segurança, previsibilidade controlada e redução de vieses estatísticos durante a seleção dos caracteres.

0845CA308DC208185F54BC3107343FA85A8E58EC0FF0B7FBF63009467DAB18897FDD41E14124951FA8BB1574BC338B93B7A932FDBF21830BD20D5911B038916B86530687BE442E69EC456101699409
25A8CD33842D6B9310DBA55B3890CD98D39324A4466B7AF051D7AE9922D44BE2E4625360B9D20B6B80648965EE3BF999ADCB0AE54210731773FF6B1729B18EFADE8F271066BC25975B447963628E4
98CA8709A864038682B7AE0E6D58341974DDCA1A9A5BDF4C1CD17E5933AEF033CD9FD5BAF40FAB209C649911CF0BB6DC7E3C158B09B9CABDC58327053C706BB030D1CEFF637BF2F94866BC8575F36
6E2955253DE595007091A7882391B8A63CD2C472057C5EF1FA9D84A18DA513DB6B8B7B3CCE4037FCE834D9135949EEC39EB0D8C0ADEEF233B9F827D37DA088E3EA125E4DF4E4AC0A6EEEB7BBD6C2E4
C81ADC2BAECF6365EA221A5299D8AE7A51F3A67EAA5AB4D776DE07321A00E604838F225BD57EA2D219F8AC0B6FE54DF3F1B514A8054065B98F37C5466AB4F888007D6B0063E679326839DC9C8DE6A8
884DC33BA5139A46FE8FF010DF5CA9AA522758D27E5E19E601449F7C8715F6B08FA814F5D601E5B79C4DD5C0BD898F3B095EC9D018E49C1E2F6A11F10AEA2AEE0A2C0CAD9EF5922AD5FEE216B7BE5B
B6B15037136E1649AFE6EB42762A99FEE302EF1E748BC2D96BFF863EF62DB26F4ACEA6BF29BEDEF82E697D5C0BE7B08921BF38995BB9744290CD3B3B852E8BB1A57B35C468ED4F28478DF88E748FA
C5FB996AA735C977CB760A92D7DBEBF9B7C44F83D837BF8323036536E26C7169B7757F44BC62B7F33B79FFBAF1ABE28CAF83E03976013324B2A213CE5618EE9DEF2B1E4DE0DAEE909B1DAEB36FC389
139F31328337CB33EA60FAABD50952826E371212ECB82D68E7196E9C3A60E197D1D2D234C471C47CD1EB633BBB1F5F933B29A1FCC47B3730736035BE51A939647BAB4D964182E9A888EB9AB1D20FC
FF334160B1C5E6C7B14887D7412BA68FBF8675D0B6885A2265E743EA617CEA110B014C00670A8F7B3D58134359F491F318F5B84DD578C0A8EA4A3D5A8A85C3D2C7E97628F192BA97242463C142C5AA
AF0565A99527E10D4F9B1805CEDBA1EC51F86DBD3B0D2123460F2A40E2DF6ABD54A4AE09FBD8E98C873E999C9BB524FA52BDAC8F9D4975AA41792D449A1A9E830F2A30C9964D8DBADBB8CAECD0181B4
D5883EFD0FA1443DEB06733F613443F24648CF51E303A958DF8FCB45130E04D5BA470BA0EFC91C73150F342793D56D6B3EEA31DCFC848DD3CDE2483C17875245F3F1A556AA7F3E60B9564E901D69C
5084BA3E53057CD7580D96242C26B687D7EBFED2684E3064D826A0D9350A3CF9531A0D0C81BDA33ECFA8663FE814D2C4369D5EF64EEDE6566FCEE9AC609DD1F38AAE25A6714D3B10770596B76E06F2

Escopo e Objetivos do Projeto

Embora o Abaddon seja capaz de gerar senhas extremamente fortes para uso direto por usuários finais, este não é o foco principal do projeto em seu estado atual. A utilização de senhas longas e de alta entropia normalmente exige o suporte de um gerenciador de senhas para armazenamento e recuperação segura das credenciais, funcionalidade que ainda não faz parte do escopo desta implementação.

Dessa forma, o projeto deve ser compreendido principalmente como uma demonstração prática de geração criptograficamente segura de credenciais, fundamentada em conceitos modernos de entropia, aleatoriedade e teoria da informação.

O principal diferencial do Abaddon está em sua arquitetura simples, modular e facilmente extensível. A lógica de geração foi projetada para separar a origem da entropia, o cálculo do universo de caracteres e o processo de seleção dos símbolos, permitindo a manutenção e expansão do sistema com relativa facilidade.

Mapa visual de funcionamento:

```
Usuário escolhe conjuntos
↓
Struct Conjunto[]
↓
calcular_universo()
↓
N = soma dos conjuntos habilitados
↓
bits_necessarios(N)
↓
bits = menor valor tal que  $2^{\text{bits}} \geq N$ 
↓
mascara = (1ULL << bits) - 1
↓
getrandom()
↓
buff[]
↓
pool_acumulador
↓
Consumir bits do buffer
↓
valor = pool_acumulador & mascara
↓
pool_acumulador >>= bits
↓
```

A struct de conjunto armazena todos os dados referentes aos valores de cada conjunto.

Conjunto é o tipo de caractere escolhido pelo usuário, número, letra, alfanumerico e etc; Essa struct armazenara dados sobre o comprimento total, o próprio conjunto de universo, o nome e se está habilitado para uso. O universo é a soma do total de caracteres de cada conjunto que pode ser representado por N.

Identificado o tamanho do universo a ser tratado para geração da senha, é passada como parâmetro para função de bits, necessários que aplica uma formula matematica de 2^{bits} para determinar quantas casas binárias possui o número que será retornado.

A mascara recebera os bits em uma variável unsigned longlong de 64 caracteres possíveis representada em bits -1.

A aleatoriedade foi colocada para ser gerada somente após todas essas etapas do código pois solicita-la no começo fatia com que valores matematicamente sensíveis flutuassem nos processos do computador aguardando decisões do usuário.

A partir desse momento o código segue uma escovação de bits para garantir que cada um seja dividido em 6 bits. E o motivo é o aproveitamento máximo da entropia gerada e maior aproximação das grandezas que serão mostradas mais a frente.

```

↓
Consumir bits do buffer
↓
valor = pool_acumulador & mascara
↓
pool_acumulador >>= bits
↓
Rejection Sampling
(valor >= N ? descarta : continua)
↓
obter_caractere()
↓
Mapeia índice global → caractere real
↓
senha[pos_senha]
↓
Repete até tamanho_senha
↓
'\0'
↓
Senha final

```

Continuando o raciocínio algorítmico, o rejection Sampling é a parte do código que valida se existirá problemas relacionados ao valor gerado. Pois se o usuário escolher um conjunto alfanumérico que contem 62 caracteres, este é o tamanho do universo definido. Mas ao transformar a análise da geração final da senha de byte a byte para bit a bit, $2^{(bits)}$ seria = a 64, então, caso o valor seja maior, o Rejection Sampling é responsável por descartar estes números restantes. Para obter o caractere, o algoritmo faz uma varredura de índice pelo caractere real para aplica-lo no vetor da senha gerada. Este processo se repete até que o tamanho definido pelo usuário seja completamente validado e o laço seja interrompido.

Gerar bytes aleatórios com `getrandom()` e mapear esses bytes para um conjunto explícito de caracteres permitido (charset).

Não usar "ASCII completo" indiscriminadamente.

Por quê?

A fórmula da entropia de uma senha gerada aleatoriamente é:

$$H = x \cdot \log_2(N)$$

onde:

- H = entropia em bits
- x = comprimento da senha
- N = tamanho do conjunto de caracteres possíveis

O que importa é o tamanho efetivo do conjunto de caracteres (N) e a uniformidade da escolha.

Exemplo

- Apenas números: $N = 10$
- Letras minúsculas: $N = 26$
- Maiúsculas + minúsculas + números: $N = 62$
- ASCII imprimível: aproximadamente $N = 95$

Fundamentos

- Utilização de mecanismos modernos de geração de números aleatórios do sistema operacional.
- Entropia derivada de eventos físicos coletados pelo hardware.
- Dependência de fontes reais de aleatoriedade para inicialização do gerador criptográfico.
- Segurança fundamentada em princípios utilizados por sistemas operacionais modernos e aplicações de segurança profissionais.

O nome **Abaddon** faz referência à entidade associada ao abismo e à provação. Da mesma forma, o projeto tem como objetivo criar senhas capazes de resistir aos testes mais rigorosos de segurança, desafiando ataques computacionais através da combinação entre alta entropia, comprimento elevado e geração criptograficamente segura.

* Todo o princípio teórico do projeto se deu pelo estudo da hive-system sobre o poder computacional necessário para se quebrar uma senha. Dentro da pasta imagens no projeto, se concentram de mais variados tipos testes que fizeram e uma extensão busca por fundamentos técnicos para o desenvolvimento do projeto para além de um

simples pedaço de código para completar uma task. Porém um trabalho real que mostra o processo correto de se inicializar uma senha segura por este MVP. Não se recomenda ainda a utilização do projeto para fins profissionais pois necessita de revisão técnica e forma adequada de armazenamento segura.